

ভারতীয় প্রযুক্তিবিদ্যা প্রতিষ্ঠান খড়্গপুর
भारतीय प्रौद्योगिकी संस्थान खड़गपुर
INDIAN INSTITUTE OF TECHNOLOGY KHARAGPUR

Transformers and the Attention Mechanism

From Sequence Modeling to Modern Large Language Models

Dr. Animesh Chaturvedi

Assistant Professor: **IIT Kharagpur**

Young Researcher: **Heidelberg Laureate Forum**

and **Pingala Interaction in Computing**

Young Scientist: **Lindau Nobel Laureate Meetings**

Postdoc: **King's College London** & **The Alan Turing Institute**

PhD: **IIT Indore** MTech: **IIITDM Jabalpur**

Introduction to Sequence Modelling with Attention and RNN

The Sequence Modelling Problem

Consider:

"The animal did not cross the road because it was tired."

What does "it" refer to?

The model needs to connect information across different positions in the sentence.

Language understanding often requires relationships between tokens that may be far apart in the sequence.

The Key Idea of Attention

Instead of compressing the entire previous sequence into one hidden state, attention allows a model to ask:

"Which other tokens are relevant to the token I am processing?"

For example:

"The cat sat on the mat because it was comfortable."

The representation of "it" can assign high attention to "cat".

Attention creates direct information pathways between sequence elements.

Attention as Information Retrieval

A useful intuition is to think of attention as a retrieval mechanism.

- The current token creates a **query**.
- Other tokens provide **keys**.
- Their associated information is stored in **values**.
- Similarity between a query and keys determines which values should receive more weight.

"I love you",
'I' is aligned with 'je', 'love' is aligned with 'aime', and 'you' is aligned with 't'
An English-French alignment matrix:

	I	love	you
je	0.94	0.02	0.04
t'	0.11	0.01	0.88
aime	0.03	0.95	0.02

Query, Key and Value: An Analogy

Imagine entering a library.

- **Query:** What information am I looking for?
- **Key:** What does each book/topic contain?
- **Value:** What information will I actually retrieve?

The query is compared against keys.
The resulting relevance scores determine how much information is retrieved from each value.
Therefore:

$$\boxed{\text{Query} + \text{Keys} \rightarrow \text{Attention Weights} \rightarrow \text{Values}}$$

Attention is therefore a learned, content-dependent information retrieval mechanism.

Before Transformers: RNNs

A recurrent neural network processes a sequence step by step:

$$h_t = f(x_t, h_{t-1})$$

where:

- x_t is the current input,
- h_{t-1} is the previous hidden state,
- h_t is the current hidden state.

Conceptually:

$$x_1 \to h_1 \to x_2 \to h_2 \to x_3 \to h_3 \to \cdots$$

The sequence is processed sequentially.

Problems with RNN-Based Sequence Modeling

RNNs face several challenges:

- Sequential computation limits parallelization.
- Long sequences can create vanishing or exploding gradients.
- Information from distant positions must pass through many recurrent steps.
- Training becomes slower as sequence length increases.

LSTMs and GRUs improve memory handling, but they still fundamentally process sequences recurrently.

Can we directly connect every token to every relevant token?

Transformer with Attention

Why Do We Need Transformers?

Many AI problems involve sequences:

- Natural language: words form sentences.
- Time series: observations occur over time.
- Speech: acoustic signals evolve sequentially.
- Source code: tokens form structured programs.
- DNA and proteins: biological sequences contain long-range dependencies.

The central challenge is understanding relationships between elements of a sequence.

Ashish Vaswani, et al. "Attention Is All You Need" NeurIPS 2017.

Attention Is All You Need

Ashish Vaswani* Google Brain avaswani@google.com

Noam Shazeer* Google Brain noam@google.com

Niki Parmar* Google Research nikip@google.com

Jakob Uszkoreit* Google Research uszr@google.com

Llion Jones* Google Research llion@google.com

Aidan N. Gomez* † University of Toronto aidan@cs.toronto.edu

Łukasz Kaiser* Google Brain lukaszkaiser@google.com

Illia Polosukhin* ‡ illia.polosukhin@gmail.com

Abstract

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles, by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.0 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature.

Ashish Vaswani

Article Talk

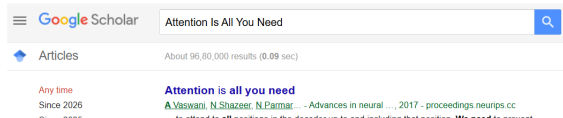
From Wikipedia, the free encyclopedia

Ashish Vaswani is an Indian computer scientist and entrepreneur. He conducted research at Google Brain, co-founded Adept AI, and co-founder and chief executive officer of Essential AI.[2][3][4] As of June 2026, Vaswani, along with Essential AI, was acqui-hired by Nvidia.[1]

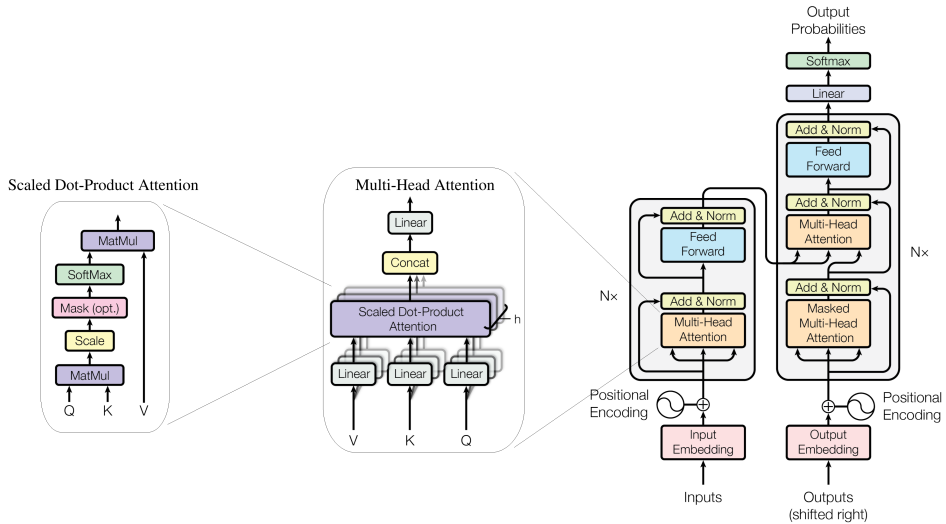
Vaswani is a co-author of the 2017 paper "Attention Is All You Need", which introduced the Transformer neural network architecture.[5] The Transformer model has been used in the development of subsequent NLP models BERT, ChatGPT, and their successors.



Ashish Vaswani	
Born	1986 (age 39–40) India
Education	University of Southern California (PhD) Birla Institute of Technology, Mesra (B.Tech.)
Known for	Transformer (deep learning architecture)
	Scientific career
Fields	Natural Language Processing Deep Learning Artificial Intelligence
Workplaces	Google Brain (2016-2021) Nvidia (2026-present)[1]
Thesis	*Smaller, Faster, and Accurate Models for Statistical Machine Translation* (2014)
Doctoral advisor	David Chiang Liang Huang



Transformer - model architecture



Why Is Position Important?

Attention by itself does not inherently know whether:

"Dog bites man"

is different from:

"Man bites dog."

Both contain the same tokens but in different orders.

Therefore, Transformers need positional information.

How Does Attention Measure Similarity?

Goal: Determine which tokens are most relevant to a given token.
Suppose the sentence is:

"The animal did not cross the road because it was tired."

To interpret the word **"it"**, the model should identify relevant contextual words, "animal".
Attention compares a query vector from one token with key vectors from other tokens to compute relevance scores.

$$S = QK^T$$

- Q: Query vectors – what each token is looking for.
- K: Key vectors – what information each token offers.
- K^T: Transpose of the key matrix.
- S: Matrix of query–key dot-product scores.

Important: These scores measure alignment, not physical distance.

Step 1: Compute Query-Key Similarity

The first operation is:

$$S = QK^T$$

Each element of S measures the compatibility between a query and a key.
For a single query:

$$s_i = q^T k_i$$

A larger dot product indicates greater similarity in the learned representation space.

The similarity matrix tells the model which tokens are potentially relevant to one another.

Step 1: Compute Query-Key Similarity

Dot Product and Cosine Similarity

For two vectors q and k, their dot product is

$$q \cdot k = \sum_{i=1}^{d_k} q_i k_i$$

The cosine similarity is

$$\boxed{\cos(\theta) = \frac{q \cdot k}{\|q\| \, \|k\|}}$$

where θ is the angle between the two vectors.

Angle	Cosine similarity
0°	1 (same direction)
60°	0.5 (partially aligned)
90°	0 (perpendicular)
180°	-1 (opposite directions)

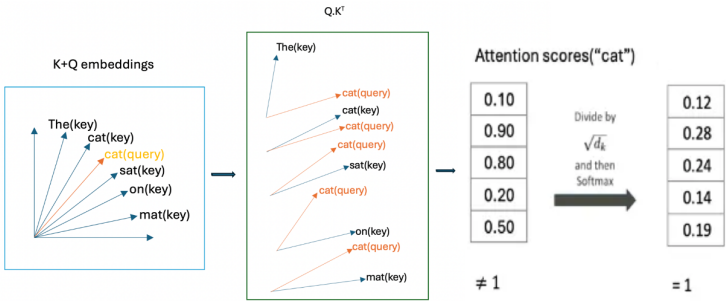
Step 1: Compute Query-Key Similarity

Connection to Attention

If both vectors are normalized to unit length, then

$$\|q\| = \|k\| = 1 \quad \Longrightarrow \quad q \cdot k = \cos(\theta).$$

Otherwise, the dot product also depends on vector magnitudes.



Step 2: Why Divide by $\sqrt{d_k}$?

Attention uses:

$$\frac{QK^T}{\sqrt{d_k}}$$

If the dimensionality d_k becomes large, dot products can also become large.
Large values entering the softmax can make its output extremely concentrated.
This can produce very small gradients for many positions.

Scaling keeps the numerical range of attention scores more stable.

Step 2: Why Divide by $\sqrt{d_k}$?

From Similarity Scores to Attention Weights

$$q = \begin{bmatrix} 1 & 0 \end{bmatrix}, \qquad k_1 = \begin{bmatrix} 1 & 0 \end{bmatrix}, \qquad k_2 = \begin{bmatrix} 0 & 1 \end{bmatrix}$$

$$qk_1^T = (1)(1) + (0)(0) = 1$$

$$qk_2^T = (1)(0) + (0)(1) = 0$$

$$\boxed{S = \begin{bmatrix} 1 & 0 \end{bmatrix}}$$

Interpret the scores

- q and k_1 point in the same direction: cosine similarity $= 1$.
- q and k_2 are perpendicular: cosine similarity $= 0$.

Let $d_k = 2$:

$$\frac{S}{\sqrt{2}} = \begin{bmatrix} 1/\sqrt{2} & 0 \end{bmatrix}$$

Step 3: Apply Softmax

Attention scores are converted into weights:

$$A = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)$$

For one query:

$$\alpha_i = \frac{e^{s_i}}{\sum_j e^{s_j}}$$

Therefore:

$$\sum_i \alpha_i = 1$$

and

$$0 \leq \alpha_i \leq 1$$

Softmax converts similarity scores into a probability distribution of attention.

Step 3: Apply Softmax

For the previous example, let $d_k = 2$:

$$\frac{S}{\sqrt{2}} = \begin{bmatrix} 1/\sqrt{2} & 0 \end{bmatrix}$$

Applying softmax gives approximately

$$A = \begin{bmatrix} 0.67 & 0.33 \end{bmatrix}$$

Interpret the scores

- The first key receives approximately 67% of the attention.
- The second key receives approximately 33%.
- Softmax produces non-negative weights that sum to 1.

Why Scale by $\sqrt{d_k}$ and Softmax?

As the key/query dimension increases, dot products can become large in magnitude. Scaling helps keep softmax inputs in a useful range and reduces the risk of saturated softmax outputs.

Step 4: Weighted Sum of Values

The final operation is:

$$Z = AV$$

For one query:

$$z = \sum_i \alpha_i v_i$$

Thus, the output is a weighted combination of the value vectors.

The model retrieves more information from highly relevant tokens.

Putting It Together: Scaled Dot-Product Attention

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

1. **Compare:** Compute QK^T to measure query–key alignment.
2. **Scale:** Divide scores by $\sqrt{d_k}$.
3. **Normalize:** Apply softmax to obtain attention weights.
4. **Aggregate:** Multiply the weights by V to combine information from the value vectors.

Remember

QK^T uses dot products; it does not automatically compute cosine similarity.

Cosine similarity is obtained when query and key vectors are normalized.

Attention weights indicate how information is combined; they are not necessarily calibrated measures of semantic similarity.

Worked Example: Calculating $S = QK^T$

Suppose we have three tokens: *The*, *cat*, and *sleeps*. We use two-dimensional vectors.

$$Q = \begin{bmatrix} 1 & 0 \\ 1 & 1 \\ 0 & 1 \end{bmatrix}, \qquad K = \begin{bmatrix} 1 & 0 \\ 1 & 1 \\ 0 & 1 \end{bmatrix} \qquad K^T = \begin{bmatrix} 1 & 1 & 0 \\ 0 & 1 & 1 \end{bmatrix}$$

Multiply Q and K^T

$$S = QK^T = \begin{bmatrix} 1 & 0 \\ 1 & 1 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 1 & 0 \\ 0 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 1 & 0 \\ 1 & 2 & 1 \\ 0 & 1 & 1 \end{bmatrix}$$

Example calculation: $S_{2,2} = (1)(1) + (1)(1) = 2$.
Interpretation: Each entry measures the dot-product alignment between one token's query and another token's key.

Worked Example: Dot Product vs. Cosine Similarity

Consider the query for the second token:

$$q_2 = \begin{bmatrix} 1 & 1 \end{bmatrix}$$

and the key vectors:

$$k_1 = \begin{bmatrix} 1 & 0 \end{bmatrix}, \quad k_2 = \begin{bmatrix} 1 & 1 \end{bmatrix}, \quad k_3 = \begin{bmatrix} 0 & 1 \end{bmatrix}$$

Step: Compute dot-product scores

$$q_2 k_1^T = 1, \qquad q_2 k_2^T = 2, \qquad q_2 k_3^T = 1$$

Therefore, the score row is

$$S_2 = \begin{bmatrix} 1 & 2 & 1 \end{bmatrix}, \qquad d_k = 2$$

Step: Scale the scores

$$\frac{S_2}{\sqrt{2}} = \begin{bmatrix} 0.707 & 1.414 & 0.707 \end{bmatrix}$$

Worked Example: From Scores to Attention Weights

Step: Apply softmax

$$\alpha_j = \frac{e^{s_j/\sqrt{2}}}{\sum_{\ell=1}^{3} e^{s_\ell/\sqrt{2}}}$$

Using $e^{0.707} \approx 2.028$ and $e^{1.414} \approx 4.113$:

$$\alpha_1 = \frac{2.028}{2.028 + 4.113 + 2.028} \approx 0.248$$

$$\alpha_2 = \frac{4.113}{8.169} \approx 0.504, \qquad \alpha_3 \approx 0.248$$

$$\boxed{A_2 \approx \begin{bmatrix} 0.248 & 0.504 & 0.248 \end{bmatrix}}$$

Conclusion

The second token receives approximately 50.4% of the attention weight because its key has the highest dot-product score for this query. The weights sum to approximately 1.

The Five Equations to Remember

1. Query, Key and Value:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V$$

2. Attention scores:

$$S = \frac{QK^T}{\sqrt{d_k}}$$

3. Attention weights:

$$A = \text{softmax}(S)$$

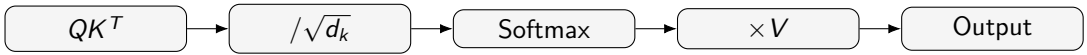
4. Attention output:

$$Z = AV$$

5. Complete attention:

$$\boxed{Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V}$$

Attention in Four Steps



$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

In languages, alignment can be multiple-to-multiple.

The English phrase "look it up" corresponds to French "cherchez-le". Thus, "soft" attention weights work better than "hard" attention weights (setting one attention weight to 1, and the others to 0).

Visualizing Self-Attention

For the sentence:

"The cat chased the mouse."

the attention matrix can conceptually look like:

$$A = \begin{bmatrix} * & * & * & * & * \\ * & * & * & * & * \\ * & * & * & * & * \\ * & * & * & * & * \\ * & * & * & * & * \end{bmatrix}$$

Each row represents a query token.
Each column represents a key token.

The matrix represents token-to-token information flow.

Self Attention Matrix

For n tokens:

$$QK^T \in \mathbb{R}^{n \times n}$$

Therefore, self-attention creates an $n \times n$ matrix.
For example, with 100 tokens:

$$100 \times 100 = 10,000$$

pairwise attention scores are computed.

This direct pairwise interaction is one reason Transformers can model long-range relationships effectively, but it also creates a significant computational cost for very long sequences.

Self-Attention vs Cross-Attention

Self-attention

$$Q, K, V \leftarrow \text{same sequence}$$

It models relationships within one sequence.

Cross-attention

$$Q \leftarrow \text{one sequence}, \qquad K, V \leftarrow \text{another sequence}$$

It allows one representation to retrieve information from another representation.

Cross-attention is important in encoder-decoder architectures.

Why One Attention Head Is Not Enough

Different relationships can exist simultaneously.
For example, a sentence may contain:

- grammatical relationships,
- semantic relationships,
- positional relationships,
- subject–verb relationships,
- entity relationships.

A single attention mechanism may not efficiently represent all these relationships.

Solution: Multi-Head Attention

Multi-Head Attention

Instead of one attention function, Transformers use multiple attention heads.
For head i:

$$head_i = Attention(QW_i^Q, KW_i^K, VW_i^V)$$

The outputs are concatenated:

$$MultiHead(Q, K, V) = Concat(head_1, \ldots, head_h)W^O$$

Each attention head can learn a different representation of relationships among tokens.

Multi-Head Attention Intuition

Imagine reading:

"The student who studied hard passed the examination."

Different heads may learn to emphasize different relationships:

- Head 1: subject relationships.
- Head 2: grammatical dependencies.
- Head 3: semantic associations.
- Head 4: nearby contextual relationships.

These are illustrative interpretations rather than guaranteed semantic roles of individual heads.

From Text to Tokens

Transformers generally operate on tokens rather than raw words.
For example:

$$\texttt{Transformers are powerful}$$

may be decomposed into token units such as:

$$[\texttt{Transform}, \texttt{ers}, \ \texttt{are}, \ \texttt{powerful}]$$

The exact tokenization depends on the tokenizer.
The pipeline becomes:

$$\text{Text} \rightarrow \text{Tokens} \rightarrow \text{Token IDs} \rightarrow \text{Embeddings} \rightarrow \text{Transformer}$$

Embeddings

Each token ID is mapped to a continuous vector:

$$\textit{token ID} \rightarrow \textit{embedding vector}$$

For example:

$$x_i \in \mathbb{R}^{d_{\text{model}}}$$

The embedding represents the token in a learned high-dimensional space.

Transformers do not directly reason over words as symbolic objects; they operate on numerical vector representations learned during training.

The Transformer Pipeline

A simplified decoder-only Transformer pipeline is:



The next token is appended to the sequence and the process repeats.

Why Transformers Scale So Well

Transformers provide several advantages:

- Attention allows direct interactions between distant tokens.
- Training can be highly parallelized across sequence positions.
- Large datasets and large models can be efficiently exploited.
- The architecture is highly compositional and stackable.
- The same basic architecture can support language, vision, audio, code, and multimodal systems.

Scalability is one of the defining strengths of Transformers.

The Computational Cost of Attention

For sequence length n:

$$QK^T$$

produces an:

$$n \times n$$

attention matrix.
Therefore, the standard self-attention computation has approximately:

$$O(n^2)$$

dependence on sequence length for the attention interaction.
Memory requirements can also become substantial for long contexts.

Long-context processing is therefore an important systems and algorithmic research problem.

Limitations of Transformers

Despite their success, Transformers have limitations:

- Quadratic attention cost with sequence length in standard self-attention.
- Large computational and memory requirements.
- High training cost for large models.
- Hallucination and factual unreliability in generative systems.
- Attention weights should not automatically be interpreted as explanations.
- Long-context performance can depend on architecture, training, and retrieval strategy.

Transformers are powerful statistical models, not guaranteed reasoning engines.

Transformers Beyond NLP

The Transformer architecture has been applied to:

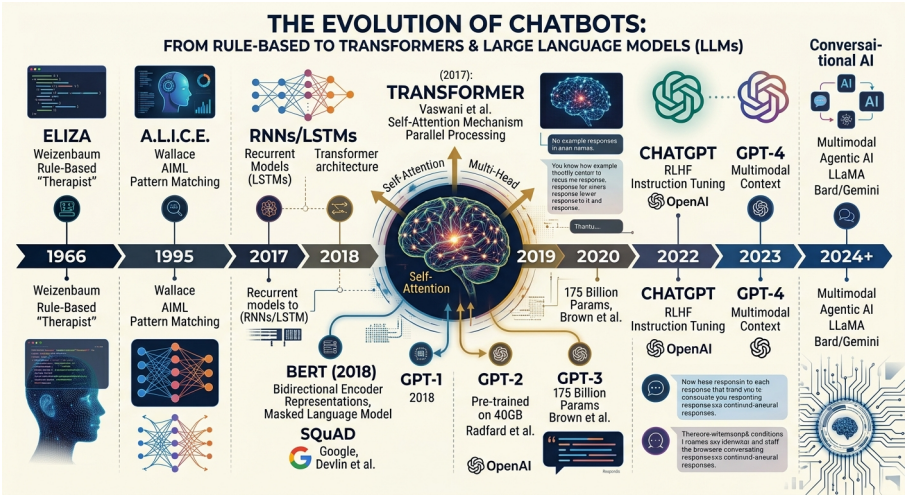
- Natural language processing
- Computer vision
- Speech processing
- Time-series forecasting
- Code generation
- Protein modeling
- Multimodal learning
- Recommendation systems

The underlying idea is general: learn relationships among elements through attention.

Applications of Transformer

Evolution of ChatBots

Rules, Patterns, RNN, LSTM, **Transformers, BERT, GPT**, Multimodal and Agentic AI



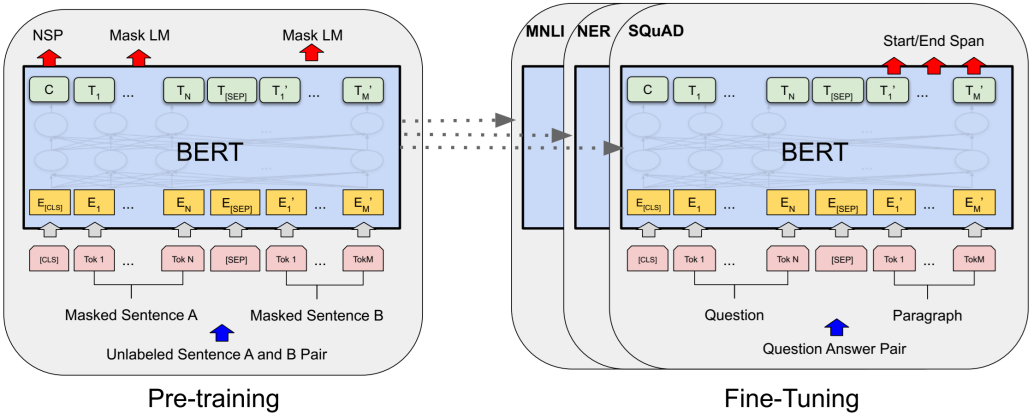
Three Important Transformer Families

Architecture	Main idea	Typical use
Encoder-only	Bidirectional contextual representation.	Classification, retrieval, representation learning
Decoder-only	Causal autoregressive generation.	Text generation and LLMs
Encoder-decoder	Input encoding followed by conditional generation.	Translation, summarization, sequence-to-sequence tasks

Modern LLMs are predominantly based on decoder-only Transformer variants.

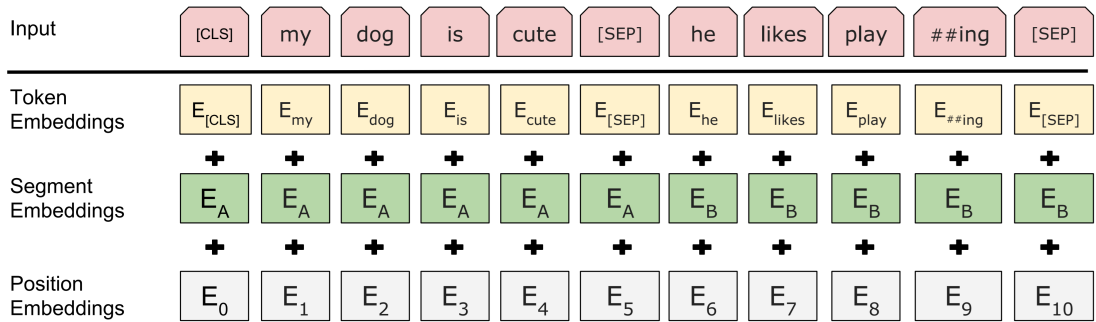
Jacob, et al. "BERT: Pre-training of Deep Bidirectional Transformers for language understanding." ACL. 2019.

Overall pre-training and fine-tuning procedures for BERT. Apart from output layers, the same architectures are used in both pre-training and fine-tuning.

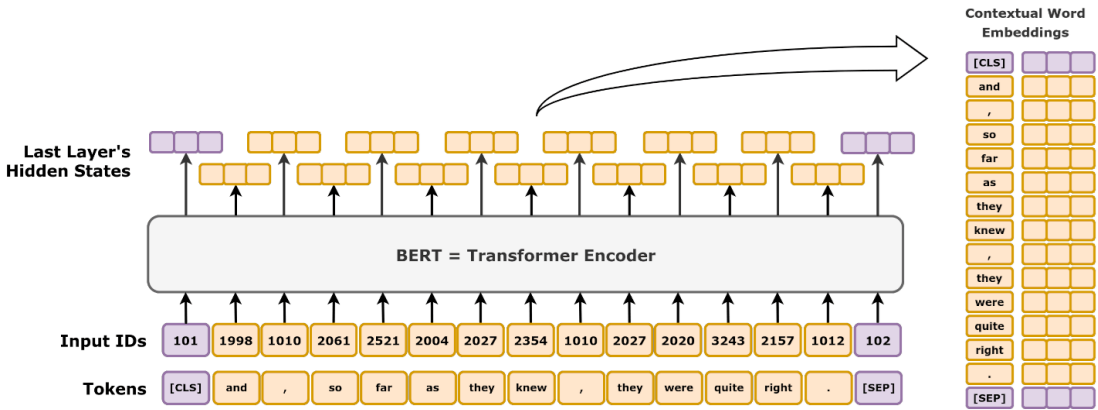


Jacob, et al. "BERT: Pre-training of Deep Bidirectional Transformers for language understanding." ACL. 2019.

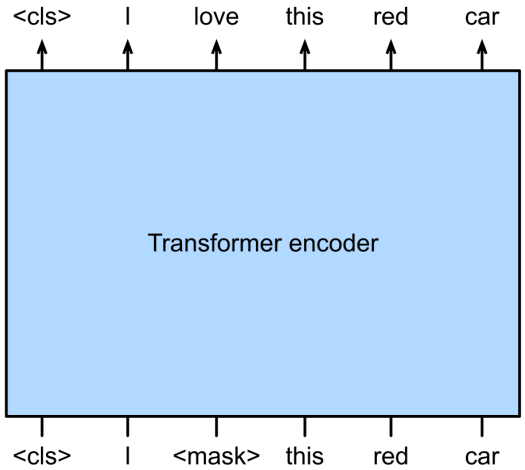
BERT input representation. The input embeddings are the sum of the token embeddings, the segmentation embeddings and the position embeddings. [CLS] is a special symbol added in front of every input example, and [SEP] is a special separator token (e.g. separating questions/answers).

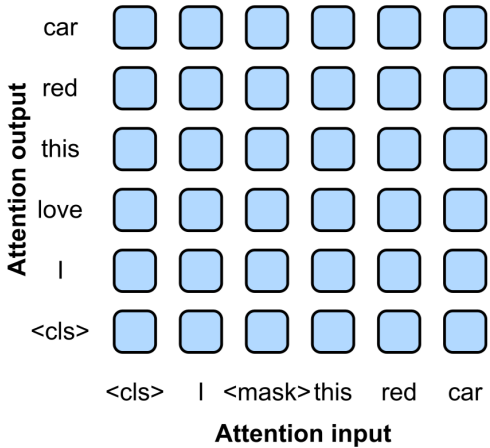


BERT: Encoder-only

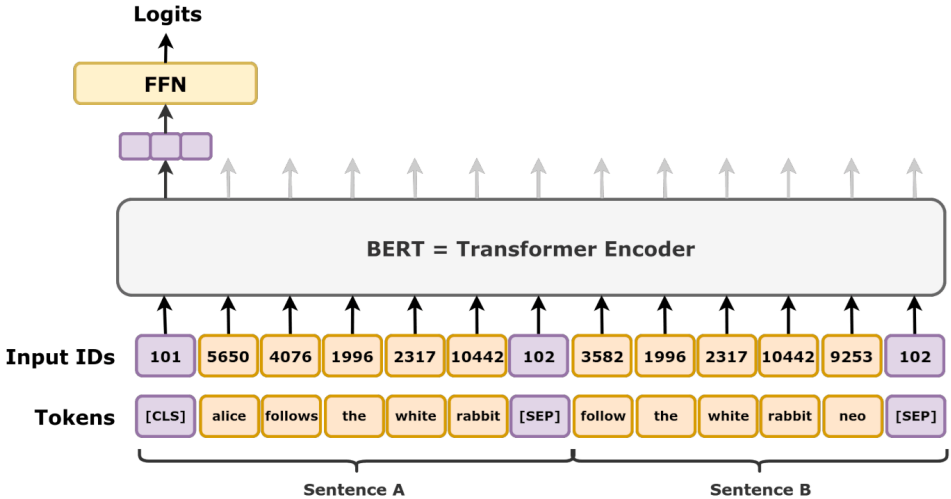


BERT: Encoder-only

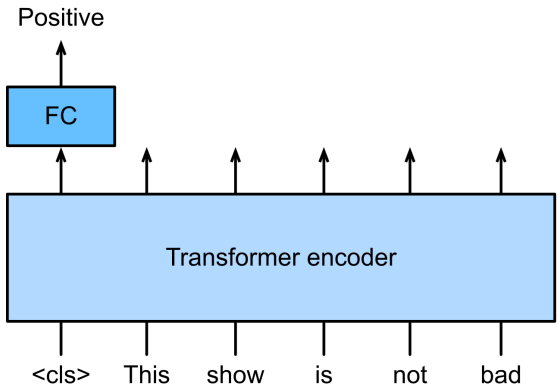




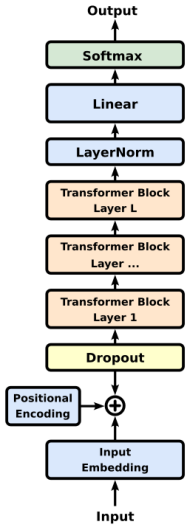
BERT: Encoder-only



BERT: Encoder-only



GPT: Decoder-only



How a Decoder-Only LLM Generates Text

Suppose the prompt is:

"The capital of India is"

The model estimates:

$$P(x_{t+1}|x_1, \ldots, x_t)$$

It selects or samples the next token.
Then the generated token becomes part of the context:

$$x_1, \ldots, x_t \rightarrow x_{t+1}$$

and the process repeats.

Token-by-token autoregressive generation

Transformers and Modern LLMs

A modern LLM can be viewed conceptually as:

$$\boxed{\textit{Tokenizer} + \textit{Embeddings} + \textit{Transformer Blocks} + \textit{Language Model Head}}$$

The Transformer provides the computational architecture.
Training provides the learned parameters.
Data provides the statistical experience.
Post-training methods shape how the model behaves.

The Transformer architecture is the foundation, but an LLM is much more than the architecture alone.

Understanding Transformers for LLMs

1. **Tokenization:** Input text is divided into tokens that can be represented numerically.
2. **Embeddings:** Each token is mapped to a high-dimensional vector that represents learned features of the token.
3. **Attention:** Tokens exchange information so that their representations can incorporate relevant context from other positions.
4. **Feed-forward layers:** Each token representation undergoes further nonlinear transformation.
5. **Repeated Transformer blocks:** Attention and feed-forward operations progressively refine token representations.
6. **Next-token prediction:** The final representation is used to produce scores and probabilities for possible next tokens.

Observe: The representation of a token changes as it passes through the network. The model converts its final representation into a prediction of the next token.

LLM Flow

Remember the following sequence:

Text $\rightarrow$ Tokens

$\downarrow$

Embeddings + Position $\rightarrow$ Self-Attention

$\downarrow$

Feed-Forward Transformation

$\downarrow$

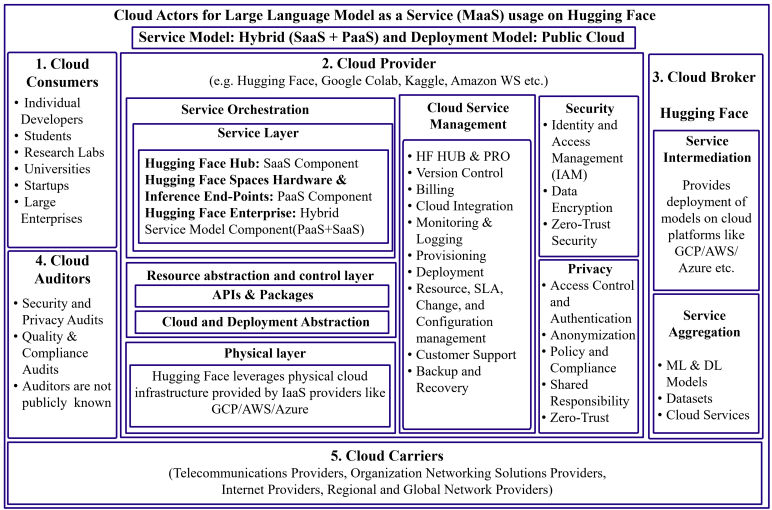
Repeated Transformer Blocks

$\downarrow$

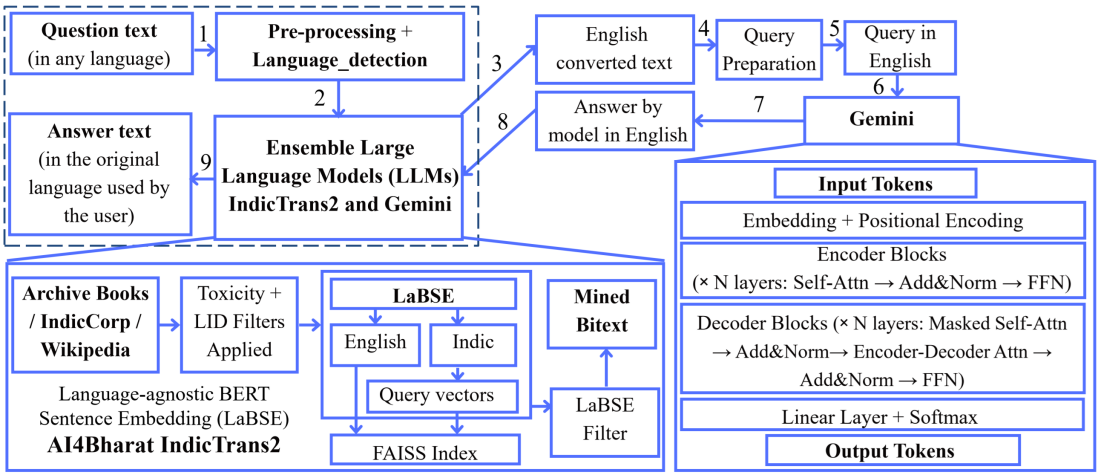
Output Distribution / Prediction

Related contributions by my research group

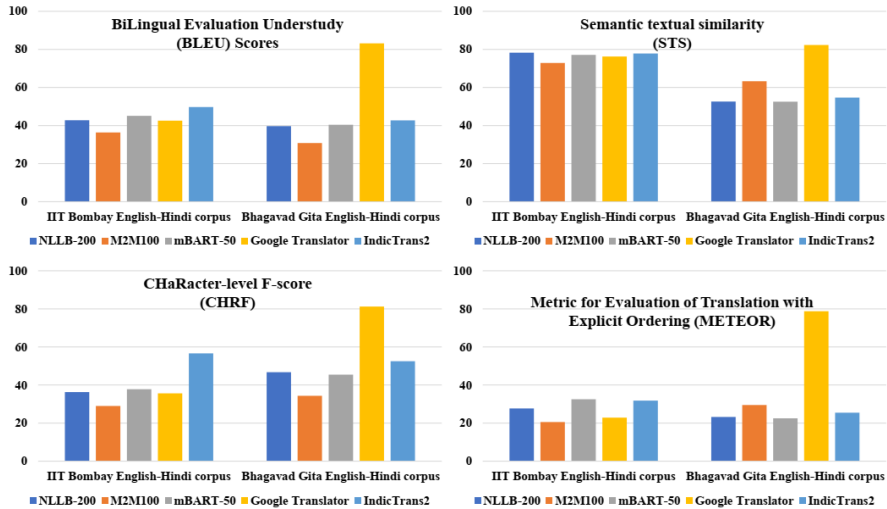
Cloud Architecture for Language Translation Model as a Service (MaaS) on Hugging Face. 5th AI-ML-Systems.



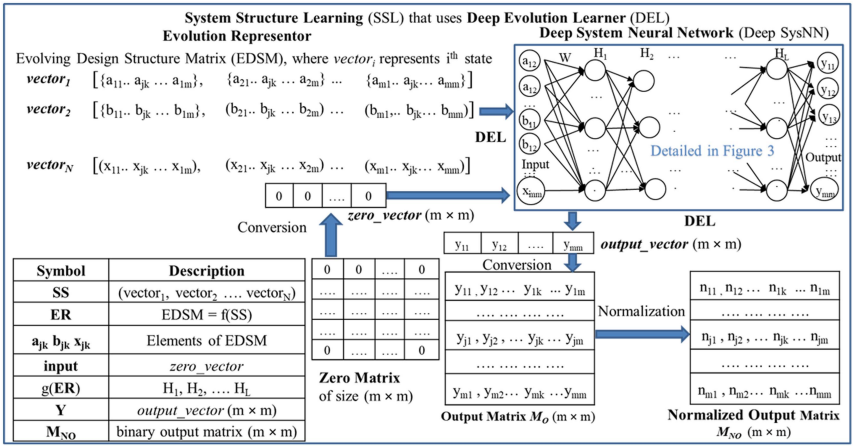
Saarthi: Ensemble AI4Bharat and Gemini for Multilingual Indian Chatbot. 5th AI-ML-Systems. IEEE, 2025. Bangalore.



“Cloud Architecture for Language Translation Model as a Service (MaaS) on Hugging Face". 5th AI-ML-Systems.



“System Neural Network: Evolution and Change based Structure Learning”. IEEE Transactions on AI, 2022.



Symbol	Description
SS	(vector$_1$, vector$_2$ …. vector$_N$)
ER	EDSM = f(SS)
a_{jk} b_{jk} x_{jk}	Elements of EDSM
input	*zero_vector*
g(ER)	H_1, H_2, …. H_L
Y	*output_vector* (m × m)
M_{NO}	binary output matrix (m × m)

Here, the ‘jk’ represents connection between two entities ‘j’ and ‘k’.

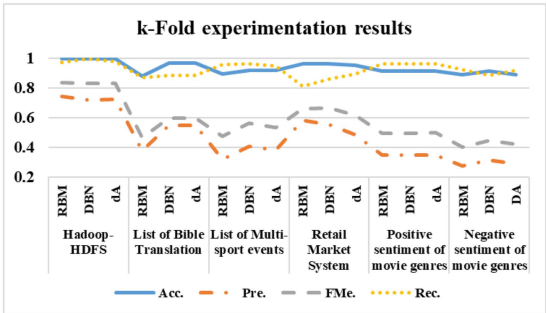
The a, b, x denotes input states, y denotes outputs, and n denotes normalized values.

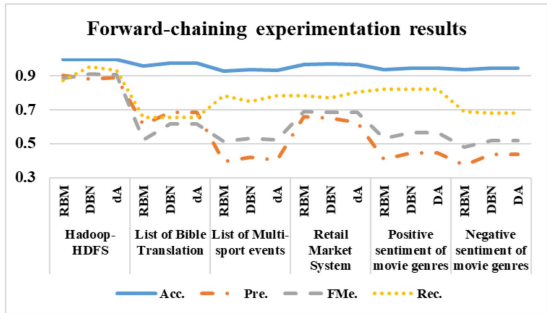
Series of ‘a’, ‘b’ and ‘x’ represent 1st, 2nd and Nth state. The L is the number of hidden layers H with weight set W.

The a_{jk} , b_{jk} , x_{jk} , n_{jk} are either 1 or 0 depending upon connection exist or not. The y_{jk} is the range between 0 to 1 such that $0 \le y_{jk} \le 1$

The EDSM size is (m×m) × N, where the m is the total number of entities in all the (N+1) states and N is number of training states.

“System Neural Network: Evolution and Change based Structure Learning”. IEEE Transactions on AI, 2022.





Conclusions

Final Perspective

Why did the Transformer change AI?

Because it provided a scalable mechanism for learning:

"Which parts of the input should influence each other?"

This simple question can be implemented through:

$$\boxed{QK^T \rightarrow Softmax \rightarrow V}$$

Key Takeaways

1. Attention allows a model to determine which parts of a sequence are relevant to one another.
2. Self-attention uses the same sequence to construct queries, keys, and values.
3. Scaled dot-product attention consists of similarity, scaling, softmax, and weighted aggregation.
4. Positional information is necessary because attention alone does not encode sequence order.
5. Multi-head attention allows different learned projections to capture different relationships.
6. Transformer blocks combine attention, feed-forward networks, normalization, and residual connections.
7. Encoder-only, decoder-only, and encoder-decoder architectures serve different purposes.
8. Modern LLMs predominantly use decoder-only Transformer architectures.


תודה רבה
Hebrew
Ευχαριστώ
Greek
Danke
German
Спасибо
Russian
धन्यवादः
Sanskrit
Merci
French
ধন্যবাদ
Bangla
நன்றி
Tamil
شكراً
Arabic
ಧನ್ಯವಾದಗಳು
Kannada
Thank You
English
നന്ദി
Malayalam
Grazie
Italian
ధన్యవాదాలు
Telugu
આભાર
Gujarati
多謝
Traditional Chinese
ਧੰਨਵਾਦ
Punjabi
धन्यवाद
Hindi & Marathi
Gracias
Spanish
多谢
Simplified Chinese
https://sites.google.com/site/animeshchaturvedi07
Obrigado
Portuguese
ありがとうございました
Japanese
ขอบคุณ
Thai
감사합니다
Korean